

LUXURY RESORT MANAGEMENT & BOOKING PLATFORM

Comprehensive Technical Documentation, System Architecture, Workflow Blueprint & Source Code Guide

Document Version:	1.0.0 (Production Release)
Core Framework:	Next.js 16 (App Router + Turbopack)
UI & Styling:	React 19, Tailwind CSS, GSAP 3, Lucide Icons
Backend & ORM:	Prisma ORM with PostgreSQL Infrastructure
Payment Gateway:	Dodo Payments API (Checkout & Automated Refunds)
Authentication:	NextAuth.js (JWT + Credentials + Nodemailer Verification)
Author / System Architect:	Senior Full-Stack Engineering Team
Date Generated:	August 8, 2026

Executive Summary:

This document contains the complete end-to-end technical documentation for the **Bookme.com Online Resort Management System**. It provides full architectural blueprints, data models, user flows, API specs, role permission matrices, and source code reference covering Guest Booking, Staff Operations, Admin Controls, and the Dynamic Tiered Cancellation Refund Engine.

1. System Architecture & Tech Stack

Bookme.com is engineered as a high-performance, responsive web application using **Next.js 16** with the App Router paradigm. The system utilizes client-side rendering with GSAP parallax animation for rich visual experiences alongside robust server-side API routes for data integrity.

Layer	Technology Selection	Key Responsibilities
Frontend Framework	Next.js 16 + React 19	App Router, Server Components, Dynamic Routing
UI & Styling	Tailwind CSS + GSAP 3	Luxury Dark Aesthetics, Micro-animations, Mouse Tilt
Maps & GIS	Leaflet.js + OpenStreetMap	Interactive resort map with flyTo destination zoom
Database & ORM	PostgreSQL + Prisma ORM	Relational schema, transaction management, seeds
Authentication	NextAuth.js + bcryptjs	JWT sessions, password hashing, role protection
Email Dispatch	Nodemailer SMTP	Verification codes, booking receipts, cancellation emails
Payments & Refunds	Dodo Payments REST API	Hosted checkout sessions, automated refund processing

2. Database Schema & Data Models

The database layer is managed via **Prisma ORM** connected to PostgreSQL. Below is the entity specification:

Model	Key Fields & Types	Relations & Role
User	id (UUID), email, password, fullName, role, isVerified	Root account entity. Supports GUEST, STAFF, ADMIN
Guest	id, phone, passportNum, userId (FK User)	Stores guest profile details & reservation history
Staff	id, employeeId, position, deptId, roleId	Linked to Department and operational Role (RBAC)
Resort	id, name, location, latitude, longitude, rating	Contains Rooms, Reviews, and Add-on Services
Room	id, roomNum, status, resortId, roomTypeId	Status: AVAILABLE, OCCUPIED, MAINTENANCE, CLEANING

Reservation	id, checkIn, checkOut, status, totalAmount	Status: PENDING, CONFIRMED, CANCELED, CHECKED_IN
Payment	id, amount, method, status, reservationId	Status: PENDING, COMPLETED, FAILED, REFUNDED
Role	id, name, permissions (String[])	Stores RBAC permission keys for staff operators

3. Authentication & Security Subsystem

Registration & 6-Digit Email Code Verification:

Guest registration is handled via `src/app/api/signup/route.ts`. When a user registers:

1. Password is hashed using `bcryptjs` (salt rounds = 10).
2. A random 6-digit numeric verification code is generated.
3. **Unverified Account Re-Registration Support:** If an unverified user submits signup again, their password and verification code are updated seamlessly instead of failing.
4. Verification emails are dispatched via `src/lib/mailer.ts`. In development environments without valid SMTP credentials, codes fall back to console logging so registration completes smoothly.

NextAuth Credentials Provider & JWT Session:

Authentication sessions are signed into secure HTTP-only cookies via NextAuth JWT provider (`src/lib/auth.ts`). Token callbacks attach the user's role (GUEST, STAFF, ADMIN) and user type so API routes enforce strict authorization checks.

4. Dynamic Tiered Cancellation & Refund Engine

The booking engine features a fair, transparent **Dynamic Tiered Cancellation Policy** with a maximum fee cap of **10%**. Guests are guaranteed to receive at least 90% of their money back upon cancellation prior to or on check-in date.

Notice Window	Refund %	Retention Fee %	Policy Description
≥ 7 Days Notice	100% Full Refund	0% Fee	Full refund guarantee. Zero retention penalty.
3 to 7 Days Notice	95% Refund	5% Processing Fee	Minor processing fee retained.
< 3 Days / Same Day / Past	90% Refund	10% Max Fee Cap	Capped at 10% maximum fee retention.

Cancellation Preview Modal & Execution Workflow:

- Dashboard Modal** (`src/app/dashboard/page.tsx`): Clicking 'Cancel Booking' opens a modal showing check-in countdown, policy tier badge, total paid amount, retention fee, and net refund amount.
- Agreement Disclaimer**: The user must explicitly check *'I have read and accept the cancellation fee policy terms'* to unlock the confirmation button.
- Backend Execution** (`DELETE /api/book`): Computes the exact net refund, triggers automated Dodo Payments API refund, updates DB reservation to CANCELED, and sends a breakdown email receipt.

Code Implementation Snippet (`DELETE /api/book`):

```
// Dynamic Tiered Refund Calculation in src/app/api/book/route.ts
const checkInTime = new Date(reservation.checkIn).getTime();
const daysUntilCheckIn = (checkInTime - Date.now()) / (1000 * 60 * 60 * 24);

let refundPercent = 100;
if (reservation.status === 'CONFIRMED') {
  if (daysUntilCheckIn >= 7) {
    refundPercent = 100; // 100% Full Refund ($0 fee)
  } else if (daysUntilCheckIn >= 3) {
    refundPercent = 95; // 95% Refund (5% fee)
  } else {
    refundPercent = 90; // 90% Refund (10% max fee cap)
  }
}
const totalPaid = Number(reservation.totalAmount);
const refundAmount = (totalPaid * refundPercent) / 100;
const retentionFee = totalPaid - refundAmount;
```

5. Payment Gateway Integration (Dodo Payments)

Bookme.com integrates with **Dodo Payments API** for hosted checkout sessions and automated refunds. The integration comprises three key components:

- 1. **Session Initialization** (`/api/checkout/[id]/session``): Converts total invoice amount into cents and requests a hosted payment checkout URL.
- 2. **Webhook Listener** (`/api/webhooks/dodo``): Listens for payment.succeeded events to automatically transition draft reservations to CONFIRMED status.
- 3. **Automated Refunds** (`/api/book``): Dispatches POST requests to `https://test.dodopayments.com/refunds` with payment ID and net refund amount.

6. Role-Based Dashboards & Operational Desks

Role Desk	Tab Views & Interfaces	Key Features & Capabilities
Guest Portal	• My Bookings • Saved Favorites • My Profile	View active/past stays, status badges, dynamic cancellation modal, resort bookmarks, and profile details.
Staff Desk	• Housekeeping Queue • Task Directory • Room Lookup	Filter room statuses (CLEAN, DIRTY, IN_PROGRESS, INSPECTED), update task assignments, and look up guest stays.
Admin Console	• Bookings Audit • Properties • Roles & Permissions • Staff Directory • Finance Ledgers	Create/edit resort properties, configure granular RBAC permissions (e.g. FINANCE_ACCESS, HOUSEKEEPING), manage staff, and trigger CSV exports.

7. Complete API Route Reference Matrix

Method	Endpoint Path	Role Protection	Functionality Description
GET	/api/resorts	Public	Fetch resort listing with pagination & location filters
GET	/api/resorts/[id]	Public	Fetch resort detail, room categories & add-on services
POST	/api/signup	Public	Register new user, hash password & dispatch code
POST	/api/signup/verify	Public	Verify 6-digit email code & activate user account
POST	/api/book	Guest	Create draft stay reservation with selected room/services
DELETE	/api/book	Guest / Staff	Cancel stay, execute tiered refund & send receipt
GET	/api/checkout/[id]	Guest	Fetch checkout summary & invoice breakdown
POST	/api/checkout/[id]/session	Guest	Generate Dodo Payments checkout session URL
POST	/api/webhooks/dodo	System	Webhook listener for payment completion events
GET	/api/dashboard/guest	Guest	Fetch guest reservations, favorites, and profile
GET	/api/dashboard/staff	Staff	Fetch housekeeping queue & operational task list
GET	/api/dashboard/admin	Admin	Fetch system KPIs, financial ledgers, & audit logs
GET/POST	/api/favorites	Guest	Fetch and toggle bookmarked resort IDs
POST	/api/admin/roles	Admin	Create custom operational role with permission array
POST	/api/admin/staff	Admin	Create staff account & assign department/role
POST	/api/admin/departments	Admin	Create new resort operational department
POST	/api/admin/services	Admin	Create new resort add-on service experience

8. Build Verification & Deployment Readiness

Automated Build Verification Output:

- **TypeScript Compiler Check** (`npx tsc --noEmit`): Clean compilation with **0 errors**.

- **Next.js Production Build (`npx next build`):** Compiled and optimized **32 static & dynamic routes** in 4.1 seconds with zero build warnings.
- **Git Remote Synchronization:** All changes committed and pushed to CodeFaisalDev/Resort-Booking-Webapp (branch `main`).

End of Official Technical Documentation — Bookme.com System Architecture